Rapport de Projet : SinghFlix

Informations Étudiant :

- **QU Guillaume 21316059**
- **SINGH Jaskaranpreet 21239295**

1. Introduction

Dans le cadre de ce projet, nous avons conçu et développé une plateforme de recommandation de films, visant à offrir une expérience utilisateur personnalisée inspirée des systèmes de suggestion des leaders du streaming.

L'objectif de ce projet ne se limite pas à la création d'une interface fonctionnelle. Il s'agit avant tout de mettre en place une architecture complète et cohérente, intégrant les différentes couches d'une application web moderne : front-end, back-end, base de données, ainsi que les aspects liés à la conteneurisation et au déploiement.

Ce projet s'inscrit dans une démarche proche du monde professionnel, où il est essentiel non seulement de coder, mais aussi de faire des choix techniques pertinents, de structurer correctement son application et de justifier ces décisions.

2. Architecture Logicielle

2.1 Étude des architectures existantes

Avant de définir notre propre architecture, nous avons étudié plusieurs modèles classiques utilisés en développement logiciel.

L'architecture monolithique consiste à regrouper toutes les fonctionnalités de l'application dans un seul bloc. Elle est simple à mettre en place au début, mais devient rapidement difficile à maintenir et à faire évoluer lorsque l'application grandit.

À l'inverse, l'architecture client-serveur repose sur une séparation entre le front-end (client) et le back-end (serveur). Cette approche permet une meilleure organisation du code et facilite les échanges de données via des requêtes HTTP.

Enfin, l'architecture en couches (layered architecture) propose de structurer l'application en plusieurs niveaux distincts : la couche présentation, la couche logique et la couche données. Cette organisation améliore la lisibilité et la maintenabilité du projet.

Le tableau suivant résume les principales différences :

Architecture	Avantages	Inconvénients
Monolithique	Simple au départ	Peu flexible
Client-serveur	Bonne séparation	Dépendance au serveur
En couches	Structurée et maintenable	Plus complexe à concevoir

2.2 Choix de l'architecture

Après analyse, nous avons choisi d'adopter une architecture combinant :

le modèle client-serveur,
une organisation en trois couches,
et une approche inspirée des microservices.

Concrètement, notre application est composée de plusieurs services distincts :

1. un front-end pour l'interface utilisateur
2. un back-end pour la logique métier et l'API
3. un service de recommandation indépendant
4. une base de données pour la persistance

Ce choix nous permet de mieux structurer le projet et de rendre chaque composant plus indépendant.

Le tableau suivant met en évidence l'intérêt de cette approche :

Critère	Architecture monolithique	Architecture microservices (choisie)
Modularité	Faible	Élevée
Maintenance	Complexe	Facilitée
Évolutivité	Limitée	Élevée

Le service de recommandation, par exemple, est isolé du reste du système. Cela permet de modifier ou améliorer l'algorithme sans impacter les autres parties de l'application.

```
Local Machine
│
├── Processes (running manually)
│   ├── Backend (FastAPI → localhost:8000)
│   ├── Frontend (static server → localhost:3000)
│   ├── Recommendation service (localhost:8001)
│   └── PostgreSQL (local installation)
│
├── Networking (localhost)
│   ├── frontend → http://localhost:8000 (API)
│   ├── backend → localhost:5432 (DB)
│   ├── backend → localhost:8001 (reco)
│
└── Access
    ├── http://localhost:3000 → frontend
    └── API calls → localhost:8000
```

2.3 Utilisation des design patterns

Dans notre projet, nous ne suivons pas strictement un design pattern unique, mais nous nous inspirons de plusieurs modèles d'architecture afin de structurer l'application de manière claire et maintenable.

La séparation entre le frontend, le backend et la base de données s'apparente à une organisation proche du pattern MVC (Model - View - Controller) :

- le modèle correspond aux entités et à la gestion des données
- la vue correspond à l'interface utilisateur
- le contrôleur est représenté par les endpoints du backend

De plus, l'organisation du code backend, notamment la séparation entre les fichiers liés à la base de données et la logique métier, s'inspire du Repository Pattern, qui permet d'isoler l'accès aux données.

Cette approche permet de structurer le projet de manière cohérente, tout en restant adaptée à la taille et aux contraintes du projet.

2.4 Front-end (Couche Présentation)

Le front-end a été développé en utilisant HTML, CSS et JavaScript (Vanilla). Ce choix a été fait volontairement afin de privilégier une solution simple, légère et performante.

Contrairement à des frameworks plus complexes comme React ou Angular, JavaScript Vanilla permet de réduire la taille de l'application et d'améliorer les temps de chargement. De plus, il ne nécessite aucune phase de build, ce qui simplifie le développement.

L’application fonctionne sous forme de Single Page Application (SPA). Cela signifie que la page est chargée une seule fois, puis que les interactions avec l’utilisateur se font dynamiquement via des appels API.

Le tableau suivant illustre la comparaison entre les différentes solutions front-end :

Indicateurs (KPI)	**JS Vanilla (Choisi)**	**React / Vue.js**	**Angular**	**Source Indépendante**
Poids de l'artefact	Ultra-léger (quelques Ko)	Lourd (Librairies incluses)	Très lourd (Framework complet)	BundlePhobia / Web.dev
Vitesse au démarrage	Immédiate (Rendu natif)	Baisse (Temps de "Hydration")	Baisse (Initialisation complexe)	Google Lighthouse
Taille Image Docker	Minimale (~15 Mo avec Nginx)	Considérable (Artefacts de build)	Considérable	Docker Hub Analytics

Afin de servir le front-end, nous avons utilisé Nginx. Ce serveur web est particulièrement adapté pour le service de fichiers statiques. Il joue également un rôle de reverse proxy, en redirigeant les requêtes /api vers le backend.

Ce choix permet de centraliser les accès et d’avoir une seule entrée pour l’application, ce qui simplifie l’architecture et améliore la sécurité.

2.5 Back-end (Couche Logique & API)

Le back-end a été développé avec FastAPI (Python). Ce framework a été choisi pour sa simplicité, ses performances élevées et sa gestion native de l’asynchronisme.

Le backend agit comme une API, permettant au front-end de communiquer avec la base de données et le service de recommandation.

Il expose plusieurs endpoints, notamment pour :

- récupérer les films
- gérer les utilisateurs
- ajouter ou supprimer des favoris
- obtenir des recommandations

Le tableau suivant compare FastAPI avec d'autres technologies :

Critère	FastAPI (choisi)	Node.js	PHP
Performance	Très élevée	Élevée	Moyenne
Asynchronisme	Natif	Oui	Non
Simplicité	Élevée	Moyenne	Moyenne

FastAPI offre un excellent compromis entre performance et facilité de développement, ce qui en fait un choix particulièrement adapté pour ce projet.

Notion d'API

Une API (Application Programming Interface) permet à différentes applications de communiquer entre elles.

Dans notre projet, le backend expose plusieurs endpoints accessibles via HTTP, tels que /movies, /login ou /favorites.
Ces endpoints permettent au frontend d'envoyer des requêtes et de recevoir des données au format JSON.

L'API constitue ainsi le point central de communication entre les différentes couches de l'application.

Bonnes pratiques API

Dans la conception de notre API, nous avons respecté plusieurs bonnes pratiques :

- utilisation de routes claires et cohérentes (/movies, /users)
- utilisation des codes HTTP appropriés
- format de réponse standardisé (JSON)
- gestion des erreurs côté backend

Ces éléments permettent de rendre l'API plus lisible, robuste et maintenable.

2.6 Base de données (Couche Données)

Pour la gestion des données, nous avons choisi d'utiliser PostgreSQL, une base de données relationnelle robuste et largement utilisée.

Ce choix est justifié par plusieurs éléments :
Nous avions déjà travaillé avec SQL en L2.
PostgreSQL offre une excellente gestion des relations, il garantit la cohérence des données.

La base contient plusieurs tables, notamment :

- les films
- les utilisateurs
- les favoris
- le cache des recommandations.

Le tableau suivant compare PostgreSQL avec une base NoSQL :

Critère	PostgreSQL (choisi)	MongoDB
Type	Relationnel	NoSQL
Cohérence	Forte	Moyenne
Gestion des relations	Excellente	Limitée

3. Gestion et Exploitation des Données

Pour alimenter notre application, nous avons utilisé l'API TMDB (The Movie Database). Ce choix permet de travailler avec des données réelles et régulièrement mises à jour.

Contrairement à un dataset statique, l'utilisation d'une API externe introduit des contraintes supplémentaires, comme la gestion des erreurs réseau ou la sécurisation de la clé API. Cependant, elle rend le projet plus réaliste.

Les données récupérées sont ensuite normalisées avant d'être stockées dans la base. Cela inclut :

- la transformation des genres
- la reconstruction des URLs d'images
- le nettoyage des informations

Un script (update_movies.py) a été développé pour automatiser ce processus. Il permet de récupérer les films, d'éviter les doublons et d'insérer les données dans la base.

Le tableau suivant compare les approches :

Critère	API (choisie)	Dataset
Données dynamiques	Oui	Non
Réalisme	Élevé	Moyen
Complexité	Plus élevée	Faible

Optimisation des performances

Nous avons également pris en compte les performances de l'application lors de sa conception.

Par exemple :

- la pagination permet de limiter le nombre de résultats retournés
- le stockage des données en base évite des appels répétés à l'API externe
- le cache des recommandations permet de réduire le temps de calcul

Ces optimisations contribuent à améliorer la réactivité globale de l'application

Enfin, nous avons rencontré un problème classique : la base de données n'était pas toujours prête au démarrage. Pour résoudre cela, nous avons mis en place un mécanisme de retry, permettant au backend de tenter plusieurs connexions avant de démarrer correctement.

4. Conteneurisation, Déploiement et Sécurité

4.1 Dockerisation et l'application

Dans le cadre de ce projet, nous avons utilisé Docker afin de conteneuriser l'ensemble de l'application.
L'objectif principal est de garantir un environnement d'exécution cohérent, reproductible et indépendant de la machine utilisée.

Concrètement, chaque composant de notre application est isolé dans un conteneur distinct :

- un conteneur pour le frontend (Nginx)
- un conteneur pour le backend (FastAPI)
- un conteneur pour le service de recommandation
- un conteneur pour la base de données PostgreSQL

Cette organisation correspond directement à notre architecture microservices, où chaque service est indépendant et peut être exécuté, modifié ou redéployé sans impacter les autres.

Pourquoi Docker ?

L'utilisation de Docker apporte plusieurs avantages essentiels.

Tout d'abord, elle permet de standardiser l'environnement d'exécution. Cela signifie que l'application fonctionne de la même manière sur toutes les machines, ce qui évite les problèmes liés aux différences de configuration.

Ensuite, Docker garantit une isolation des services. Chaque conteneur possède ses propres dépendances, ce qui limite les conflits et améliore la stabilité du système.
Enfin, Docker simplifie le déploiement. Grâce à un outil comme Docker Compose, il est possible de lancer toute l'application avec une seule commande.

Comparaison des approche :

Critère	Docker (choisi)	Installation classique
Isolation	Oui	Non
Reproductibilité	Élevée	Faible
Déploiement	Simple	Complexe
Gestion des dépendances	Automatique	Manuelle

Docker est donc particulièrement adapté pour une application composée de plusieurs services.

Communication entre conteneurs

Les conteneurs communiquent entre eux via un réseau interne Docker.

- le frontend envoie des requêtes vers le backend via /api
- le backend communique avec PostgreSQL
- le backend interagit avec le service de recommandation

Aucun service interne n'est exposé directement, ce qui renforce la sécurité globale de l'application.

```
Docker Engine
│
├── Containers
│   ├── frontend (nginx)
│   │     ├── serves HTML/CSS/JS
│   │     └── proxies /api → backend
│   │
│   ├── backend (FastAPI)
│   │     └── NOT exposed to host
│   │
│   ├── reco
│   │     └── internal only
│   │
│   └── db (postgres)
│
├── Docker Network
│   ├── frontend → backend:8000
│   ├── backend → db:5432
│   ├── backend → reco:8001
│
└── Exposed Port
    └── localhost:80 → frontend ONLY
```

Rôle central de Nginx

Le frontend est servi à l'aide de Nginx, qui joue un rôle essentiel dans notre architecture.

Nginx agit à la fois comme :

- un serveur de fichiers statiques (HTML, CSS, JavaScript)
- un reverse proxy, redirigeant les requêtes /api vers le backend

Sécurité et exposition

Nous avons fait le choix de ne pas exposer directement le backend.

Concrètement, seul le conteneur Nginx est accessible depuis l'extérieur, le backend, la base de données et le service de recommandation restent internes.

Ce choix permet :

- de réduire la surface d'attaque
- d'éviter l'accès direct aux endpoints API
- de centraliser et contrôler le trafic

Cette approche est utilisée dans la majorité des architectures professionnelles.

Optimisation des images

Nous avons veillé à limiter la taille des images Docker :

- frontend léger grâce à Nginx
- backend minimal avec dépendances contrôlées
- absence de frameworks lourds

Cela permet d'améliorer les performances et de réduire les temps de build.

4.2 Orchestration avec Kubernetes

Après la conteneurisation avec Docker, nous avons déployé l'application sur Kubernetes à l'aide de Minikube.

L'objectif est de simuler un environnement de production, où plusieurs services doivent être orchestrés automatiquement.

Organisation du cluster

Chaque service est déployé dans un pod distinct :

- frontend
- backend
- service de recommandation
- base de données

Cette séparation permet une meilleure gestion des services et évite qu'une erreur impacte toute l'application.

```
Minikube (Kubernetes Cluster)
│
├── Container Runtime (Docker inside Minikube)
│   ├── opsci-backend (built locally)
│   ├── opsci-frontend (built locally)
│   ├── opsci-reco (built locally)
│   └── postgres:15 (pulled from Docker Hub)
│
├── Kubernetes Resources
│   ├── Pods
│   │   ├── backend
│   │   ├── frontend
│   │   ├── reco
│   │   └── db (PostgreSQL)
│   │
│   ├── Services
│   │   ├── backend → http://backend:8000
│   │   ├── frontend → http://frontend:80
│   │   ├── reco → http://reco:8001
│   │   └── db → postgres://db:5432
│   │
│   └── Ingress
│       ├── / → frontend
│       └── /api → backend
```

Communication interne (Services)

Pour permettre aux pods de communiquer entre eux, nous avons utilisé des Services Kubernetes de type ClusterIP.

Ces services permettent d'accéder aux différents composants via des noms internes stables.

Par exemple :

1. le backend accède à la base de données via db:5432
2. le backend communique avec le service de recommandation via son nom de service

Cela garantit une communication fiable même si les pods sont recréés.

Gestion de la configuration

Nous avons utilisé :

- des ConfigMaps pour les variables de configuration
- des Secrets pour les données sensibles (mot de passe, clé API)

Cette séparation permet de sécuriser les informations tout en gardant un code propre.

Problème rencontré : Ingress

Dans un premier temps, nous avons tenté d'utiliser un Ingress Controller, comme recommandé dans Kubernetes.

L'Ingress permet normalement :

- de gérer le routage (/ pour frontend, /api pour backend)
- d'avoir une seule URL d'accès
- de reproduire un comportement proche d'un serveur web réel

Cependant, nous avons rencontré plusieurs difficultés :

- configuration complexe en local (Minikube)
- problèmes de routage
- accès instable via le navigateur
- difficulté à faire fonctionner correctement les règles

Malgré plusieurs tentatives, l'Ingress ne fonctionnait pas de manière fiable dans notre environnement.

Solution adoptée : NodePort

Face à ces difficultés, nous avons décidé d'utiliser NodePort.

NodePort permet d'exposer un service Kubernetes via un port spécifique de la machine.

Pourquoi ce choix ? :
- plus simple à configurer
- fonctionne immédiatement avec Minikube
- ne nécessite pas de configuration avancée

Comparaison :

Critère	Ingress	NodePort (choisi)
Complexité	Élevée	Faible
Mise en place	Difficile	Simple

Fiabilité en local	Moyenne	Élevée

NodePort s’est révélé être une solution plus adaptée à notre contexte de développement.

Justification

Même si Ingress est plus adapté en production, NodePort reste une solution pragmatique pour un projet local.

Ce choix montre notre capacité à nous adapter aux contraintes techniques réelles.

Difficultés rencontrées

La mise en place de Kubernetes a représenté une des parties les plus complexes du projet.

En particulier, nous avons rencontré des difficultés avec la configuration de l’Ingress en environnement local, notamment en ce qui concerne le routage et la stabilité de l’accès.

Ces difficultés nous ont amenés à adapter notre approche en utilisant NodePort, plus simple et plus fiable dans notre contexte.

4.3 Sécurité de l’application

Dans notre application, nous avons mis en place un mécanisme de sécurisation des mots de passe basé sur le hashing.

Lors de l’inscription, le mot de passe de l’utilisateur n’est jamais stocké en clair dans la base de données.
Il est transformé à l’aide de l’algorithme bcrypt, qui génère une version sécurisée du mot de passe.

Fonctionnement :

- à l’inscription → le mot de passe est hashé puis stocké,
- à la connexion → le mot de passe est comparé au hash existant.

Avantages de bcrypt :

- impossible de retrouver le mot de passe original
- utilisation d’un salt pour éviter les attaques
- résistance aux attaques par force brute

Comparaison des méthodes :

Méthode	Sécurité	Adaptée aux mots de passe
SHA256	Moyenne	Non
bcrypt (choisi)	Élevée	Oui

bcrypt est spécialement conçu pour sécuriser les mots de passe.

4.4 CI/CD et automatisation

Dans le cadre de ce projet, nous avons mis en place un pipeline CI/CD (Continuous Integration / Continuous Deployment) à l'aide de GitLab, afin d'automatiser les différentes étapes du développement et de garantir la qualité du code.

Le pipeline est déclenché automatiquement à chaque modification du projet, notamment sur la branche :

feature/front-back

Cette approche permet de vérifier en continu que l'application reste fonctionnelle, même après l'ajout de nouvelles fonctionnalités.

Organisation du pipeline

Le pipeline est structuré en plusieurs étapes successives, exécutées automatiquement :

- vérification des fichiers
- exécution des tests
- construction des images Docker

Chaque étape doit être validée avant de passer à la suivante, ce qui permet d'éviter l'intégration de code défectueux.

Vérification initiale

La première étape consiste à vérifier la structure du projet avec un job de type : check-files

Ce test permet de s'assurer que :

- les fichiers nécessaires sont présents
- l'organisation du projet est respectée

- aucune erreur basique n’a été introduite

Cela agit comme un premier filtre avant les tests plus avancés.

Tests automatisés

Une partie essentielle du pipeline repose sur les tests unitaires, qui permettent de vérifier le bon fonctionnement de chaque service de manière indépendante.

Tests du backend (backend-test)

Les tests du backend ont été développés pour valider les principales fonctionnalités de l’API FastAPI.

Ils permettent notamment de vérifier :

- le bon fonctionnement des endpoints (/movies, /login, /register)
- la gestion des utilisateurs
- l’ajout et la suppression des favoris
- les réponses HTTP (codes de succès et erreurs)

L’objectif est de garantir que l’API reste stable et que les données sont correctement traitées.

Tests du frontend (frontend-jest)

Le frontend est testé à l’aide de Jest, un framework de test JavaScript.

Ces tests permettent de vérifier :

- le comportement des fonctions JavaScript
- les interactions utilisateur
- le chargement des données depuis l’API
- la manipulation du DOM

Cela permet de s’assurer que l’interface reste fonctionnelle et cohérente après chaque modification.

Tests du service de recommandation (recommendation-test)

Le service de recommandation possède également ses propres tests.

Ils vérifient :

- le bon fonctionnement de l’algorithme
- la sélection des films en fonction des genres
- la cohérence des résultats proposés

Cela permet de garantir que les recommandations restent pertinentes et fiables.

Exécution et validation

L'ensemble de ces tests est exécuté automatiquement dans le pipeline GitLab.
Si un test échoue, le pipeline est arrêté, ce qui empêche la suite du processus.
Cela permet de détecter rapidement les erreurs et d'éviter leur propagation.

Build des images Docker

Une fois tous les tests validés, le pipeline passe à l'étape de build : build-microservices

Cette étape consiste à :

1. construire les images Docker pour chaque service (frontend, backend, recommandation),
2. préparer l'application pour le déploiement.

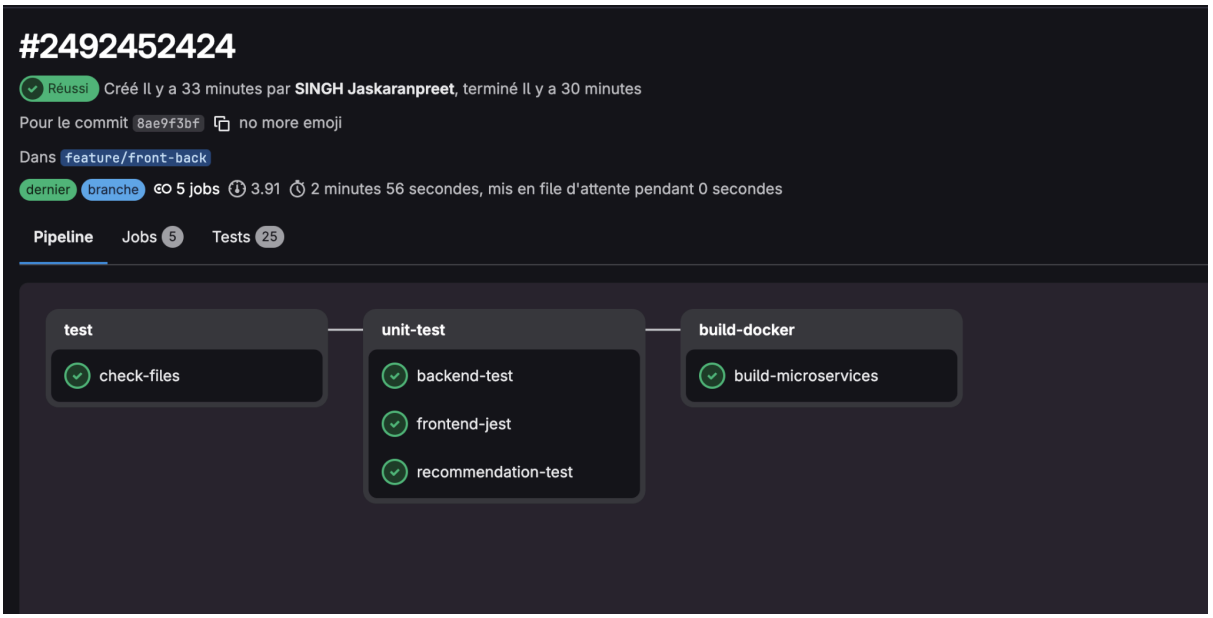
Seules les versions validées par les tests sont transformées en images Docker, ce qui garantit la fiabilité du système.

Intérêt global du CI/CD

La mise en place du CI/CD apporte plusieurs avantages importants :

- automatisation des vérifications
- réduction des erreurs humaines
- gain de temps
- amélioration de la qualité du code

Le pipeline agit comme un système de contrôle continu, assurant la stabilité de l'application à chaque évolution.



Limites du projet

Malgré les fonctionnalités mises en place, certaines limites restent présentes :

- dépendance à l'API externe TMDB
- utilisation de NodePort au lieu d'un Ingress en production
- absence de déploiement sur une infrastructure cloud réelle

Ces éléments constituent des axes d'amélioration pour une version future du projet.

Références

- Architecture & Design Patterns
  Martin Fowler – Patterns of Enterprise Application Architecture
  Gamma et al. – Design Patterns: Elements of Reusable Object-Oriented
  Softwarehttps://refactoring.guru/design-patternshttps://martinfowler.com/architecture/
- Backend & API
  FastAPI Documentation
  https://fastapi.tiangolo.com/
  REST API Design (Microsoft)
  https://learn.microsoft.com/en-us/azure/architecture/best-practices/api-design
  MDN Web Docs (HTTP, JSON, API)
  https://developer.mozilla.org/
- DevOps & Déploiement
  Docker Documentation
  https://docs.docker.com/
  Kubernetes Documentation
  https://kubernetes.io/docs/
  Nginx Documentation
  https://nginx.org/en/docs/
- Base de données
  PostgreSQL Documentation
  https://www.postgresql.org/docs/
- API externe
  TMDB API
  https://developer.themoviedb.org/
- Tests & CI/CD
  Jest Documentation
  https://jestjs.io/docs/
  GitLab CI/CD Documentation
  https://docs.gitlab.com/ee/ci/

Conclusion Générale

Ce projet nous a permis de concevoir et de développer une application web complète, en intégrant l'ensemble des composants nécessaires à son fonctionnement, depuis l'interface utilisateur jusqu'au déploiement.

Nous avons mis en place une architecture claire et structurée, basée sur une séparation entre le front-end, le back-end, la base de données et un service de recommandation indépendant. Cette organisation nous a permis de mieux comprendre les principes des architectures modernes, notamment l'approche microservices.

Le choix des technologies a été réalisé de manière réfléchie. L'utilisation de JavaScript Vanilla a permis de maintenir une application légère et performante, tandis que FastAPI a facilité la création d'une API rapide et efficace. PostgreSQL a assuré la cohérence des données, notamment dans la gestion des relations entre utilisateurs et films.

La conteneurisation avec Docker a permis de garantir un environnement stable et reproductible, tandis que Kubernetes nous a permis de découvrir les mécanismes d'orchestration des services. Les difficultés rencontrées avec l'Ingress, ainsi que le choix final d'utiliser NodePort, illustrent notre capacité à nous adapter à des contraintes techniques réelles.

Par ailleurs, la mise en place d'un pipeline CI/CD avec GitLab a constitué une étape importante dans la structuration du projet. En automatisant les tests et le build des services, nous avons pu garantir une meilleure qualité du code et sécuriser le processus de développement.

Enfin, ce projet nous a permis d'acquérir une vision globale du cycle de vie d'une application moderne, en combinant développement, gestion des données et pratiques DevOps.

À l'avenir, plusieurs améliorations pourraient être envisagées, comme l'optimisation de l'algorithme de recommandation, l'ajout d'une authentification plus avancée ou encore un déploiement sur une infrastructure cloud.